UNITY 6000.0.62F1 · COMPLETE OFFLINE DOCUMENTATION

Sandfall Blocks Developer Guide

Install. Edit. Reskin. Build your release.

szekipapa77@gmail.com



CHAPTER 01

Start here: Sandfall Blocks

Sandfall Blocks is a standalone editable Unity game template. Drag a tray piece into the jar. The piece becomes grains that fall and settle. A connected colour span touching both side walls clears. Complete the displayed span, colour, score or gem goal within the block budget; overflowing the limit or exhausting blocks before the goal loses. Rock layouts and gems change the challenge. Endless mode has its own score-based progression.

Includes 25 curated levels, level generation, a game-specific visual editor and validation, 4 guided helpers, local saving and portrait/landscape scenes. Keep the required private game Kit and publisher-owned Szekipapa77Ads layer. Provider SDK binaries are excluded. There is no bundled HLK, purchases, multiplayer server or remote LiveOps backend. Import only one game Kit into a project because the Kit classes share names.

CHAPTER 02

Requirements, installation and first play

Use Unity **6000.0.62f1**, URP **17.0.4** and Unity UI **2.0.0**. Install matching Android SDK/NDK/OpenJDK and WebGL modules in Unity Hub. Retain the supplied manifest and ProjectSettings.

1. Extract the standalone project to a short writable local path, then open it in the matching editor. Wait for package import and compilation.
2. For native unitypackage import, create a matching URP project and use Assets > Import Package > Custom Package. Import every game, Kit, Szekipapa77Ads and WebGL-template entry.
3. Run Szekipapa77 > Sandfall Blocks > Setup. It configures rendering and player settings; review its changes when integrating with existing work.
4. Active Input Handling must support the classic Input Manager: choose Old or Both. New Input System only is insufficient.
5. Open `Assets/Games/Sandfall/Scenes/Sandfall_Mobile.unity` and press Play. Follow the level card and first-level instructions.
6. Stop Play mode before editing level assets. Keep all .meta GUIDs. The game uses classic Unity UI Text; do not assume an extra TextMesh Pro import is required.

CHAPTER 03

Project folders and exact source map

All game-relative paths below start at `Assets/Games/Sandfall`. Runtime UI is built in code; editing an unrelated static Canvas does not reskin it. Scripts contains game rules and views; Editor contains authoring tools; Kit supplies UI/audio/save/platform support; Resources/sandfall contains level JSON, art, music and sound; Settings owns URP assets; Scenes contains both entry scenes.

- `Scripts/SandfallGame.cs`
- `Scripts/SandfallGame.Hud.cs`
- `Scripts/SandfallGame.View.cs`
- `Scripts/SfArt.cs`
- `Scripts/SfBoard.cs`
- `Scripts/SfBot.cs`
- `Scripts/SfCore.cs`
- `Scripts/SfGen.cs`
- `Scripts/SfLevels.cs`

Main flow: SandfallGame.cs. UI and popup consumers: SandfallGame.Hud.cs. Model/generator/loading: SfCore.cs, SfBoard.cs, SfGen.cs, SfLevels.cs. Authoring: Editor/SandfallLevelEditor.cs. Shared ad settings live outside the game at Assets/Szekipapa77Ads/Resources/AdsSettings.asset.

CHAPTER 04

Gameplay, controls and progression

Drag a tray piece into the jar. The piece becomes grains that fall and settle. A connected colour span touching both side walls clears. Complete the displayed span, colour, score or gem goal within the block budget; overflowing the limit or exhausting blocks before the goal loses. Rock layouts and gems change the challenge. Endless mode has its own score-based progression.

The initial bank has 25 curated levels. Read the supplied loader/generator before changing continuation rules. Play the guided opening, ordinary progress, completion and retry paths; preserve saved indexes during a visual reskin. The Mobile scene is portrait and the WebGL-named scene is landscape, but the supplied Build menu selects portrait for both build targets. Select landscape explicitly in your Build Profile if desired. Test a near-startup scene replacement as well as a normal restart.

CHAPTER 05

Tutorials and every helper

Fresh local progress triggers the first-level guidance. Newly unlocked helpers have introduction and guided-use flows. Reset only a test save when reviewing tutorials. A free guided first use can differ from ordinary charge consumption. BoostName, BoostUnlock, BoostStart and BoostDesc are in SandfallGame.cs. Tutorial and popup consumers are in SandfallGame.Hud.cs.

Helper	Unlock	Free charges	Action and effect
Reroll	3	3	Replaces the three tray pieces.
Scoop	6	2	Arm Scoop, then tap sand in the jar to remove a circular area.
Rainbow	10	2	Arms the next piece as rainbow sand; place the piece to see the effect.
Undo	15	2	Restores the previous jar and tray state. Needs a completed placement.

Activate the tool and perform its required target action: arming a tool alone does not demonstrate its effect. Undo needs an eligible previous move, Scoop needs sand, Spin needs a nonsymmetric comb and Drip needs a multi-cell comb. Exhausted charges may offer a rewarded ad; verify success/cancel/failure and grant a reward once only.

CHAPTER 06

Visual level editor: worked editing session

Open **Szekipapa77 > Sandfall Blocks > Level Editor**. Exact implementation: `Editor/SandfallLevelEditor.cs`.

Duplicate a level before editing. The editor exposes the jar dimensions, starting layout, colour/piece rules, goals and seed. Paint the initial jar using its brush/palette controls and keep the goal achievable within the block budget. Edit the visible name and first-time hint separately from data keys. Validate one level, Validate All, then Save and Play this level. The exact controls and current assignments are listed below; the canonical SfLevel data controls runtime behavior.

1. Back up Resources/sandfall/Levels/levels.json.
2. Duplicate a simple early level and edit one small element.
3. Use the game-specific validation controls and resolve every reported issue.
4. Save explicitly; stop and restart Play mode, then use Play this level or navigate to the edited index.
5. Check portrait and landscape and rebuild demo outputs after accepting data changes.

The exact current authoring controls, including tooltips and generator switches, follow. These are game-specific source references, not a generic editor feature list.

Source	Exact current code
`SandfallLevelEditor.cs:47`	`[MenuItem("Szekipapa77/Sandfall Blocks/Level Editor")]`
`SandfallLevelEditor.cs:50`	`[MenuItem("Szekipapa77/Sandfall Blocks/Validate All Levels")]`
`SandfallLevelEditor.cs:116`	`if (GUILayout.Button(_dirty ? "Save *" : "Save", EditorStyles.toolbarButton, GUILayout.Width(70))) Save();`
`SandfallLevelEditor.cs:117`	`if (GUILayout.Button("Reload", EditorStyles.toolbarButton, GUILayout.Width(60)) && (!_dirty || EditorUtility.DisplayDialog("Reload", "Discard unsaved changes?", "Discard", "Cancel"))) Load();`
`SandfallLevelEditor.cs:120`	`if (GUILayout.Button("Validate", EditorStyles.toolbarButton, GUILayout.Width(70)) && Cur != null) ValidateOne(_sel);`
`SandfallLevelEditor.cs:121`	`if (GUILayout.Button("Validate All", EditorStyles.toolbarButton, GUILayout.Width(85))) ValidateAll();`

SandfallLevelEditor.cs:124	`if (GUILayout.Button(new GUIContent("Play this level", "Quick test: saves, opens the WebGL scene at this level and enters Play mode. Editing never needs it."), EditorStyles.toolbarButton, GUILayout.Width(100)) && Cur != null) Play(_sel + 1);`
SandfallLevelEditor.cs:135	`if (GUILayout.Button("Add")) { var l = SfGen.Generate(_file.levels.Length + 1, 0.4f, Environment.TickCount & 0xFFFF, null, false); Insert(_file.levels.Length, l); }`
SandfallLevelEditor.cs:136	`if (GUILayout.Button("Dup") && Cur != null) Insert(_sel + 1, Cur.Clone());`
SandfallLevelEditor.cs:137	`if (GUILayout.Button("Del") && Cur != null && EditorUtility.DisplayDialog("Delete", $"Delete level {_sel + 1}?", "Delete", "Cancel"))`
SandfallLevelEditor.cs:142	`if (GUILayout.Button("Up") && _sel > 0) Swap(_sel, _sel - 1);`
SandfallLevelEditor.cs:143	`if (GUILayout.Button("Down") && _sel < _file.levels.Length - 1) Swap(_sel, _sel + 1);`
SandfallLevelEditor.cs:180	`EditorGUILayout.LabelField($"Level {_sel + 1}", EditorStyles.largeLabel);`
SandfallLevelEditor.cs:184	`L.name = EditorGUILayout.TextField("Name", L.name);`
SandfallLevelEditor.cs:185	`EditorGUILayout.LabelField("First-time hint");`
SandfallLevelEditor.cs:188	`int nli = EditorGUILayout.Popup("Layout", Mathf.Max(0, li), SfGen.Layouts);`
SandfallLevelEditor.cs:190	`int cols = EditorGUILayout.IntSlider("Columns (cells)", L.cols, 6, 13);`
SandfallLevelEditor.cs:191	`int rows = EditorGUILayout.IntSlider("Rows (cells)", L.rows, 10, 18);`
SandfallLevelEditor.cs:193	`L.colors = EditorGUILayout.IntSlider("Colours dealt", L.colors, 1, Sf.MaxColors);`
SandfallLevelEditor.cs:195	`L.shapes = ShapeTiers[EditorGUILayout.Popup("Block shapes", ti, ShapeTiers)];`

SandfallLevelEditor.cs:196	`L.pieces = EditorGUILayout.IntField("Block limit (0 = none)", L.pieces);`
SandfallLevelEditor.cs:197	`L.tray = EditorGUILayout.TextField(new GUIContent("First tray", "Optional 'shape:colour' x3, e.g. 2:1,4:3,1:2"), L.tray);`
SandfallLevelEditor.cs:198	`L.seed = EditorGUILayout.IntField("Seed", L.seed);`
SandfallLevelEditor.cs:199	`L.difficulty = EditorGUILayout.Slider("Design difficulty", L.difficulty, 0, 1);`
SandfallLevelEditor.cs:200	`L.breather = EditorGUILayout.Toggle("Breather", L.breather);`
SandfallLevelEditor.cs:202	`EditorGUILayout.LabelField("Goals (all must be met)", EditorStyles.boldLabel);`
SandfallLevelEditor.cs:203	`L.goalSpans = EditorGUILayout.IntSlider("Spans (any colour)", L.goalSpans, 0, 20);`
SandfallLevelEditor.cs:205	`L.goalColor = EditorGUILayout.Popup("Colour spans", L.goalColor, GoalColorNames);`
SandfallLevelEditor.cs:208	`L.goalScore = EditorGUILayout.IntField("Score", L.goalScore);`
SandfallLevelEditor.cs:209	`L.goalGems = EditorGUILayout.Toggle($"Free every gem ({L.CountGems()})", L.goalGems);`
SandfallLevelEditor.cs:210	`EditorGUILayout.HelpBox(L.GoalText() + (L.pieces > 0 ? $" with {L.pieces} blocks" : ""), MessageType.None);`
SandfallLevelEditor.cs:214	`if (GUILayout.Button(new GUIContent("Recompose", "New dunes, rocks and gems from this layout and seed (SfGen.Compose); keeps name, hint and goals' kinds."))) Recompose(L, false);`
SandfallLevelEditor.cs:215	`if (GUILayout.Button(new GUIContent("New seed", "Recompose with a fresh seed"))) { L.seed = UnityEngine.Random.Range(1, 99999); Recompose(L, false); }`
SandfallLevelEditor.cs:216	`if (GUILayout.Button(new GUIContent("Calibrate", "Size goals and block limit with the bot (SfGen.Calibrate)"))) Recompose(L, true);`

SandfallLevelEditor.cs:219	`EditorGUILayout.HelpBox($"{(v.solvable ? "Solvable" : "NOT SOLVABLE")}: {v.note}\nweak bots win {v.weakRate * 100:0}% difficulty {v.difficulty:0}/100", v.solvable ? MessageType.Info : MessageType.Error);`
SandfallLevelEditor.cs:275	`if (GUILayout.Button("Clear map", GUILayout.Width(80))) { for (int y = 0; y < L.DotH; y++) L.map[y] = new string('.', L.DotW); Changed(); }`
SandfallLevelEditor.cs:277	`EditorGUILayout.LabelField("Left-drag paints, right-drag erases. Each dot is half a block cell. Right: the jar after the sand settles (red line = limit).", EditorStyles.miniLabel);`
SandfallLevelEditor.cs:322	`EditorGUILayout.LabelField($"Sand dots {L.CountSand()} gems {L.CountGems()} rock dots {L.CountRocks()}", EditorStyles.miniLabel);`
SandfallLevelEditor.cs:348	`EditorGUILayout.LabelField("Difficulty curve (bar = validated difficulty if available, else design; blue = breather, red = unsolvable). Click a bar to select.", EditorStyles.miniLabel);`
SandfallLevelEditor.cs:368	`_showGen = EditorGUILayout.Foldout(_showGen, "Auto Generate (same SfGen code as the runtime)", true);`
SandfallLevelEditor.cs:371	`_genCount = EditorGUILayout.IntField("Count", _genCount, GUILayout.Width(220));`
SandfallLevelEditor.cs:372	`_genStart = EditorGUILayout.IntField("Start level #", _genStart, GUILayout.Width(220));`
SandfallLevelEditor.cs:373	`_genSeed = EditorGUILayout.IntField("Seed", _genSeed, GUILayout.Width(220));`
SandfallLevelEditor.cs:376	`_genFrom = EditorGUILayout.Slider("Start difficulty", _genFrom, 0, 1, GUILayout.Width(330));`
SandfallLevelEditor.cs:377	`_genTo = EditorGUILayout.Slider("End difficulty", _genTo, 0, 1, GUILayout.Width(330));`
SandfallLevelEditor.cs:378	`_genBreather = EditorGUILayout.IntSlider("Breather every", _genBreather, 0, 8, GUILayout.Width(330));`

`SandfallLevelEditor.cs:380`	`_genReplace = EditorGUILayout.ToggleLeft("Replace the range starting at 'Start level #' (otherwise insert there)", _genReplace);`
`SandfallLevelEditor.cs:391`	`if (GUILayout.Button($"Generate {_genCount} validated levels", GUILayout.Width(260))) Generate();`
`SandfallLevelEditor.cs:421`	`_log = $"Generated {made.Count} levels at #{at + 1} ({(_genReplace ? "replaced" : "inserted")}).";`

CHAPTER 07

Generator, insertion and replacement

Use the Auto Generate panel in SandfallLevelEditor.cs. Choose Count, start position, seed, difficulty curve and breather cadence. Review every exposed feature toggle and the Replace-range option in the exact source-control table below. Insertion keeps later records and shifts their indexes; replacement deliberately overwrites the selected range. Append after index 25, validate the resulting bank and Save. SfGen/SfBot verification is not a substitute for human play.

Worked extension: back up the bank, generate five candidates after the original 25 at moderate difficulty, validate them, then Save. Reopen the JSON and confirm the count changed. An unchecked replace option inserts rather than silently destroying later data. Runtime continuation does not automatically append new records to project JSON. Validation and bot scores help find problems but do not certify a pleasant human difficulty curve.

CHAPTER 08

Actual level JSON and identifiers

Canonical asset: `Assets/Games/Sandfall/Resources/sandfall/Levels/levels.json`. Array order controls visible numbering. Preserve numeric/boolean types, root version fields where present and resource/save identifiers. The actual first record is shown below; use the editor for structured coordinates and block/word data.


```
{
  "DotW": 16,
  "DotH": 24,
  "W": 48,
  "H": 72,
  "HasGoal": true,
  "name": "First Pour",
  "hint": "Drag a block into the jar. It melts into sand! Join one colour from wall to wall to clear it.",
  "layout": "open",
  "cols": 8,
  "rows": 12,
  "colors": 3,
  "pieces": 8,
  "goalSpans": 1,
  "goalColor": 0,
  "goalColorSpans": 0,
  "goalScore": 0,
  "goalGems": false,
  "shapes": "easy",
  "tray": "2:1,4:3,1:2",
  "seed": 11,
  "difficulty": 0.06,
  "breather": false,
  "map": [
    "................",
    "................",
    "................",
    "................",
    "................",
    "................",
    "................",
    "................",
    "................",
    "................",
    "................",
    "................",
    "................",
    "................",
    "................",
    "................",
    "................",
    "................",
    "................",
    "................",
    "1111111111......",
    "1111111111......",
    "3333333322222222",
    "3333333322222222"
  ],
  "botPieces": 1,
  "botScore": 5.6
}
```


Root keys: version, levels. Inspect SfCore.cs, SfBoard.cs, SfGen.cs, SfLevels.cs for field semantics and validation. Keep visible names/hints distinct from technical keys. For Word Hunt, words, masks and placements are gameplay data; interface translation is a separate task.

CHAPTER 09

Reskin recipe: interface colours

Stop Play mode and open `Scripts/SfArt.cs`. Find `Brass`, currently `#C9963F`. Change that hex literal to `#42C9B0`, preserve its symbol/type, compile and reopen the scene. Review HUD, menus, helper cards, settings/results and both orientations. Art sprites and procedural fallback shapes can have additional colours; edit them deliberately.

Actual consumers of this symbol are listed first, then all current palette/hex assignments. Editor-only palettes do not control the runtime. Preserve state/goal contrast and rebuild after accepting a change.

Source	Exact current code
`SandfallGame.cs:535 / void Lose(bool overflow)`	`eb.Add(("Back to Levels", SfArt.Brass, () => StartLevel()));`
`SandfallGame.cs:551 / void Lose(bool overflow)`	`buttons.Add(("Retry Level", SfArt.Brass, () => StartLevel()));`
`SandfallGame.Hud.cs:263 / void BuildLevelPill(RectTransform parent, Vector2 pos, Vector2 size)`	`pill.AddOutline(new Color(SfArt.Brass.r, SfArt.Brass.g, SfArt.Brass.b, 0.6f), 3);`
`SandfallGame.Hud.cs:291 / void BuildLevelPill(RectTransform parent, Vector2 pos, Vector2 size)`	`card.AddOutline(new Color(SfArt.Brass.r, SfArt.Brass.g, SfArt.Brass.b, 0.45f), 3);`
`SandfallGame.Hud.cs:317 / void BuildLevelPill(RectTransform parent, Vector2 pos, Vector2 size)`	`var cap = HzUI.Label("Cap", card.transform, _endless ? "ENDLESS" : "GOALS", 34, SfArt.Brass);`
`SandfallGame.Hud.cs:569 / void BuildBooster(RectTransform parent, int b, Vector2 pos, float size)`	`else { var li = HzIcons.Icon(lk.transform, HzIcons.Lock, (int)(size * 0.42f), SfArt.Brass); li.rectTransform.Place(new Vector2(0.5f, 0.5f), new Vector2(size, size * 0.5f), new Vector2(0, size * 0.1f)); }`
`SandfallGame.Hud.cs:809 / void ShowLevelCard()`	`var sub = HzUI.Label("Name", p.Card, D.name, 40, SfArt.Brass);`
`SandfallGame.Hud.cs:849 / void ShowSettingsPopup()`	`("How to Play", SfArt.Brass, () => HelpPage(0)),`
`SandfallGame.Hud.cs:909 / void HelpPage(int i)`	`Popup($"How to Play {i + 1}/{HelpPages.Length}", HelpPages[i], SfArt.Brass, (last ? "Got it!" : "Next", SfArt.Good, last ? (Action)null : () => HelpPage(i + 1)));`

SandfallGame.View.cs:61 / void BuildBoard(RectTransform parent, Vector2 center, Vector2 maxSize)	var frame = HzUI.Panel("Frame", _boardBox, SfArt.Brass, 26);
SandfallGame.View.cs:64 / void BuildBoard(RectTransform parent, Vector2 center, Vector2 maxSize)	var rim = HzUI.Panel("Rim", frame.transform, SfArt.BrassDark, 20);
SandfallGame.View.cs:247 / void BuildTray(RectTransform parent, Vector2[] pos, Vector2 slotSize)	var ring = HzUI.Panel("Ring", slot.transform, new Color(SfArt.Brass.r, SfArt.Brass.g, SfArt.Brass.b, 0.35f), 30);

Source	Exact current code
SandfallGame.cs:57 / class/static declarations	public static readonly string[] BoostColor = { "#E0803A", "#C9963F", "#9D7BFF", "#2EA7C4" };
SfArt.cs:9 / class/static declarations	public static readonly Color Night = HzUI.Hex("#1B1640"), Dusk = HzUI.Hex("#3A2A5E"), Brass = HzUI.Hex("#C9963F"), BrassDark = HzUI.Hex("#7A5520"),
SfArt.cs:10 / class/static declarations	Parchment = HzUI.Hex("#FFF4DC"), Ink = HzUI.Hex("#3A2550"), Glass = new Color(0.1f, 0.08f, 0.22f, 1f), Panel = new Color(0.13f, 0.1f, 0.27f, 0.88f),
SfArt.cs:11 / class/static declarations	Good = HzUI.Hex("#4CC38A"), Danger = HzUI.Hex("#FF4D6D"), Saffron = HzUI.Hex("#FFC145"), Coral = HzUI.Hex("#FF6F59"), Teal = HzUI.Hex("#2EC4B6"),
SfArt.cs:12 / class/static declarations	Lilac = HzUI.Hex("#9D7BFF"), Sub = HzUI.Hex("#CBB8F0"), Locked = HzUI.Hex("#4A3F6B"), Sky = HzUI.Hex("#4FA8FF");
SfArt.cs:17 / class/static declarations	Color.clear, HzUI.Hex("#FF6F59"), HzUI.Hex("#FFC145"), HzUI.Hex("#2EC4B6"), HzUI.Hex("#9D7BFF"),
SfArt.cs:18 / class/static declarations	HzUI.Hex("#FF5FA2"), HzUI.Hex("#7BE07A"), HzUI.Hex("#4FA8FF"), HzUI.Hex("#FFFFFF"), HzUI.Hex("#6E6585"),

CHAPTER 10

Reskin recipe: visible menu and HUD text

Open `Scripts/SandfallGame.Hud.cs`, find the displayed literal `"Settings"`, and replace its displayed value with `"Options"`. Keep hierarchy names and technical IDs unchanged. In HzUI.Label/Button calls the first string may be an object name; change the displayed text argument. Read the method references below to distinguish construction from later refresh assignments.

Preserve interpolation braces, invariant number formatting, ON/OFF updates and line breaks. Update both initial and refresh paths, portrait and landscape. BoostName, BoostUnlock, BoostStart and BoostDesc are in SandfallGame.cs. Tutorial and popup consumers are in SandfallGame.Hud.cs. Level names/hints remain separate JSON content. If a title appears in art, Title property or another screen, update those visible uses deliberately without renaming GameId, namespace, resources or save keys.

Hard-coded game strings do not automatically use shared localization. To translate the interface, route every display assignment through your translation table. Dictionaries, word banks and theme pools are independent gameplay assets and need separate validation. Test help, settings, results, tutorial and helper introductions after a fresh launch.

Source	Exact current code
`SandfallGame.cs:51 / class/static declarations`	`public static readonly string[] BoostName = { "Reroll", "Scoop", "Rainbow", "Undo" };`
`SandfallGame.cs:431 / void EndlessColorsCheck()`	`Toast("New colour: " + Sf.ColorNames[c] + "!", SfArt.Sand[c], 1.8f, IsPortrait ? 0.72f : 0.8f);`
`SandfallGame.cs:481 / void Win()`	`ResultPopup(done == 25 ? "Journey Complete!" : "Level Complete!", msg, SfArt.Good, "win_badge", stars, buttons.ToArray());`
`SandfallGame.cs:537 / void Lose(bool overflow)`	`HzTween.Delay(this, 0.7f, () => ResultPopup(best ? "New Best!" : "Sand Overflow!", $"Score {S.Score}\nBest {Best}", SfArt.Coral, null, 0, eb.ToArray()));`
`SandfallGame.cs:555 / void Lose(bool overflow)`	`HzTween.Delay(this, 0.7f, () => ResultPopup(overflow ? "Sand Overflow!" : "Out of Blocks!", why + "\n" + ProgressText(), SfArt.Coral, null, 0, buttons.ToArray()));`
`SandfallGame.cs:647 / void OnBooster(int b)`	`if (!Unlocked(b)) { HzAudio.Play(Sfx.Invalid, 0.5f); Toast($"{BoostName[b]} unlocks at level {BoostUnlock[b]}", SfArt.Sub, 1.3f, IsPortrait ? 0.3f : 0.2f); return; }`
`SandfallGame.cs:659 / void OnBooster(int b)`	`HzPlatform.Rewarded(GameId, "booster_" + BoostName[b].ToLowerInvariant(), ok =>`

SandfallGame.cs:667 / void OnBooster(int b)	Toast($"+1 {BoostName[b]}", SfArt.Saffron, 1.1f, IsPortrait ? 0.3f : 0.2f);
SandfallGame.cs:701 / void OnBooster(int b)	else Toast("Tap the sand to scoop it out", SfArt.Saffron, 1.4f, IsPortrait ? 0.3f : 0.2f);
SandfallGame.cs:721 / void OnBooster(int b)	if (!_undoValid) { HzAudio.Play(Sfx.Invalid, 0.5f); Toast("Nothing to undo yet", SfArt.Sub, 1.1f, IsPortrait ? 0.3f : 0.2f); return; }
SandfallGame.Hud.cs:179 / void LayoutLandscape(float rw, float rh)	var cap = HzUI.Label("TrayCap", right, "Drag a block into the jar", 34, SfArt.Sub, TextAnchor.MiddleCenter, false);
SandfallGame.Hud.cs:267 / void BuildLevelPill(RectTransform parent, Vector2 pos, Vector2 size)	_levelText = HzUI.Label("T", pill.transform, lv, 56, SfArt.Saffron);
SandfallGame.Hud.cs:273 / void BuildLevelPill(RectTransform parent, Vector2 pos, Vector2 size)	_nameText = HzUI.Label("N", pill.transform, _endless ? $"Best {Best}" : D.name, 30, SfArt.Sub, TextAnchor.MiddleCenter, false);
SandfallGame.Hud.cs:317 / void BuildLevelPill(RectTransform parent, Vector2 pos, Vector2 size)	var cap = HzUI.Label("Cap", card.transform, _endless ? "ENDLESS" : "GOALS", 34, SfArt.Brass);
SandfallGame.Hud.cs:347 / void BuildLevelPill(RectTransform parent, Vector2 pos, Vector2 size)	var t = HzUI.Label("Text", r, "", row ? 44 : 46, Color.white, TextAnchor.MiddleLeft);
SandfallGame.Hud.cs:367 / void BuildLevelPill(RectTransform parent, Vector2 pos, Vector2 size)	_piecesText = HzUI.Label("N", pc, "", 64, Color.white);
SandfallGame.Hud.cs:371 / void BuildLevelPill(RectTransform parent, Vector2 pos, Vector2 size)	_piecesCaption = HzUI.Label("C", pc, "blocks", 28, SfArt.Sub, TextAnchor.MiddleCenter, false);
SandfallGame.Hud.cs:463 / void BuildStats(RectTransform parent, Vector2 pos, Vector2 size)	_piecesText = HzUI.Label("N", l, "", 58, Color.white); _piecesText.rectTransform.Place(new Vector2(0.5f, 0.5f), new Vector2(size.x / 2, 64), new Vector2(0, 16));
SandfallGame.Hud.cs:465 / void BuildStats(RectTransform parent, Vector2 pos, Vector2 size)	_piecesCaption = HzUI.Label("C", l, "blocks left", 28, SfArt.Sub, TextAnchor.MiddleCenter, false); _piecesCaption.fontStyle = FontStyle.Bold;

`SandfallGame.Hud.cs:471 / void BuildStats(RectTransform parent, Vector2 pos, Vector2 size)`	`_scoreText = HzUI.Label("N", r, "0", 58, SfArt.Saffron); _scoreText.rectTransform.Place(new Vector2(0.5f, 0.5f), new Vector2(size.x / 2, 64), new Vector2(0, 16));`
`SandfallGame.Hud.cs:473 / void BuildStats(RectTransform parent, Vector2 pos, Vector2 size)`	`var sc = HzUI.Label("C", r, "score", 28, SfArt.Sub, TextAnchor.MiddleCenter, false); sc.fontStyle = FontStyle.Bold;`
`SandfallGame.Hud.cs:485 / void BuildStatsCompact(RectTransform box)`	`_scoreText = HzUI.Label("N", box, "0", 56, SfArt.Saffron, TextAnchor.MiddleLeft);`
`SandfallGame.Hud.cs:491 / void BuildStatsCompact(RectTransform box)`	`var sc = HzUI.Label("C", box, "score", 28, SfArt.Sub, TextAnchor.MiddleLeft, false); sc.fontStyle = FontStyle.Bold;`
`SandfallGame.Hud.cs:543 / void BuildBooster(RectTransform parent, int b, Vector2 pos, float size)`	`v.Name = HzUI.Label("Name", bg.transform, BoostName[b], 28, Color.white);`
`SandfallGame.Hud.cs:555 / void BuildBooster(RectTransform parent, int b, Vector2 pos, float size)`	`v.BadgeText = HzUI.Label("N", v.Badge.transform, "", (int)(size * 0.24f), Color.white); v.BadgeText.rectTransform.Stretch();`
`SandfallGame.Hud.cs:571 / void BuildBooster(RectTransform parent, int b, Vector2 pos, float size)`	`v.LockText = HzUI.Label("Lv", lk.transform, "Lv " + BoostUnlock[b], (int)(size * 0.2f), SfArt.Saffron);`
`SandfallGame.Hud.cs:599 / void RefreshBoosters()`	`v.BadgeText.text = _guide == b ? "1" : c > 0 ? c.ToString() : "+";`
`SandfallGame.Hud.cs:627 / void RefreshHud()`	`_piecesText.text = S.Unlimited ? "∞" : S.PiecesLeft.ToString();`
`SandfallGame.Hud.cs:635 / void RefreshHud()`	`if (_nameText && _endless) _nameText.text = $"Best {Mathf.Max(Best, S.Score)}";`
`SandfallGame.Hud.cs:651 / void RefreshHud()`	`case 0: text.text = $"{Mathf.Min(S.Spans, D.goalSpans)}/{D.goalSpans} spans"; done = S.Spans >= D.goalSpans; break;`
`SandfallGame.Hud.cs:653 / void RefreshHud()`	`case 1: text.text = $"{Mathf.Min(S.ColorSpans, D.goalColorSpans)}/{D.goalColorSpans} {Sf.ColorNames[D.goalColor].ToLowerInvariant()}"; done = S.ColorSpans >= D.goalColorSpans; break;`

SandfallGame.Hud.cs:655 / void RefreshHud()	case 2: text.text = $"{Mathf.Min(S.GemsFreed, S.GemsTotal)}/{S.GemsTotal} gems"; done = S.GemsFreed >= S.GemsTotal; break;
SandfallGame.Hud.cs:657 / void RefreshHud()	case 3: text.text = $"{Mathf.Min(S.Score, D.goalScore)}/{D.goalScore}"; done = S.Score >= D.goalScore; break;
SandfallGame.Hud.cs:659 / void RefreshHud()	default: text.text = $"{S.Spans} spans"; done = false; break;
SandfallGame.Hud.cs:785 / void ShowLevelCard()	var p = ArtPopup($"Level {Level}", msg, SfArt.Lilac, art, 0, ("Play", SfArt.Good, (Action)Begin));
SandfallGame.Hud.cs:809 / void ShowLevelCard()	var sub = HzUI.Label("Name", p.Card, D.name, 40, SfArt.Brass);
SandfallGame.Hud.cs:827 / void UnlockPopup(int b, Action done)	ArtPopup("New: " + BoostName[b] + "!", BoostDesc[b] + $"\nYou get {BoostStart[b]} free!", HzUI.Hex(BoostColor[b]), SfArt.Get(BoostArt[b]), 0, ("Great!", SfArt.Good, done));
SandfallGame.Hud.cs:849 / void ShowSettingsPopup()	("How to Play", SfArt.Brass, () => HelpPage(0)),
SandfallGame.Hud.cs:861 / void ShowSettingsPopup()	var p = Popup("Settings", "", SfArt.Lilac, list.ToArray());
SandfallGame.Hud.cs:881 / static string SoundLabel() => HzAudio.Muted ? "Sound: OFF" : "Sound: ON";	static string SoundLabel() => HzAudio.Muted ? "Sound: OFF" : "Sound: ON";
SandfallGame.Hud.cs:883 / static string MusicLabel() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";	static string MusicLabel() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";
SandfallGame.Hud.cs:909 / void HelpPage(int i)	Popup($"How to Play {i + 1}/{HelpPages.Length}", HelpPages[i], SfArt.Brass, (last ? "Got it!" : "Next", SfArt.Good, last ? (Action)null : () => HelpPage(i + 1)));
SandfallGame.Hud.cs:931 / void BuildGuideLayer()	_tipText = HzUI.Label("T", _tip, "", 38, Color.white, TextAnchor.MiddleCenter, false);
SandfallGame.Hud.cs:1181 / void EndGuide(bool used)	if (used) Toast("Nice! Boosters refill with rewarded ads.", SfArt.Saffron, 1.6f, IsPortrait ? 0.3f : 0.18f);

`SandfallGame.View.cs:472 / void GemFx(int n)`	`Toast(n == 1 ? "Gem freed!" : $"{n} gems freed!", SfArt.Saffron, 1.0f, IsPortrait ? 0.68f : 0.82f);`
`SandfallGame.View.cs:483 / void FloatText(Vector2 at, string text, Color c, int size, float life = 0.9f)`	`var l = HzUI.Label("Float", _fxLayer, text, size, c);`

CHAPTER 11

Artwork, fonts, audio and licences

Replace game images under Resources/sandfall through Unity while retaining resource keys, .meta GUIDs, Sprite/UI import settings, alpha, pivots and slicing. Procedural graphics in the art/view sources may need separate colour changes. Replacing one PNG does not change every fallback. Keep third-party font and other licence notices with the corresponding shipped assets.

Music is Resources/sandfall/Music/dune_loop.ogg. Preserve its load key when replacing it, check loop boundaries/compression and test Sound and Music toggles separately. Kit UI uses supplied fonts; validate glyph coverage for new languages and retain the actual font licence files.

CHAPTER 12

Local saves, reset and migration

HzSave wraps PlayerPrefs and flushes local state on focus loss/quit. WebGL persistence uses browser storage tied to the hosting origin. Clearing site data or changing origin starts a new local save; this is not cloud synchronization. Inspect HzSave/PlayerPrefs calls in the actual game source before altering progression, stars, helper charges or tutorial flags.

Verify fresh launch, a completed puzzle, helper use, sound options, focus loss and restart. Use a dedicated test profile for resets. Keep technical save keys stable during colour/text reskins. If changing schema or keys, implement and test a migration rather than silently discarding existing users' progress.

CHAPTER 13

Advertising integration and configuration

Asset: `Assets/Szekipapa77Ads/Resources/AdsSettings.asset`. Inspect backend, useGoogleTestIds and missingSdkFallback in the publisher-owned Ads configuration. Buyer projects include adapter/integration code and exclude external provider SDK binaries. WebGL and a missing-provider fallback simulate ads; these are not live commercial impressions.

For real Android ads install the supported provider SDK from its official source into your working project, enter your own app/placement IDs and use test ads first. Verify banner/interstitial/rewarded callbacks on supported hardware. For production use your own IDs and disable simulated missing-SDK fallback. Build tooling rejects unsafe production fallback settings. No IAP, remote backend or HLK is included in this publisher edition.

CHAPTER 14

Android build and release

Install Unity 6000.0.62f1 Android modules through Hub. Select Android, run Setup and review application ID `com.szekipapa77.sandfall`, version, scene orientation and signing. The build command writes `Builds/Android/sandfall.apk`. Confirm BuildResult.Succeeded and the actual output file; intermediate report totals are not the final APK size.

Use your own keystore/signing and distribution settings. A test-signed demo is not a signed store release. A provider-free build simulates ads; a provider test-demo does not prove production ad-account configuration. Test real hardware touch, safe areas, focus/audio, save/relaunch and reward callbacks. Keep passwords, keystores and private service credentials out of buyer packages.

CHAPTER 15

WebGL build and hosting

Install matching WebGL support. Run Setup and Build WebGL; output is `Builds/WebGL/sandfall/`. Serve index.html with all loader/framework/data/WASM/StreamingAssets files over HTTP(S), not file://. Match compression, Content-Encoding and MIME types to the host. Outer page sizing and actual scene orientation are different settings.

The publisher's playable demo is hosted on HyperCodeStore. Upload the actual runtime files rather than merely changing a URL. Test a fresh anonymous load, real pointer controls, audio after interaction, saved progress/reload and phone/desktop sizing. Rebuild and rehost when code/assets change. A successful HTTP page alone is not gameplay verification.

CHAPTER 16

Troubleshooting

Compile errors: restore matching packages and the full private Kit/Ads layer; only one game Kit belongs in a project. **Pink/invisible rendering:** run Setup and inspect Graphics and every Quality URP assignment. **No input:** classic input handling must be enabled; check canvas focus and blocking modals. **Missing resources:** restore exact paths, .meta GUIDs and importer settings.

Edits do not appear: stop Play, Save in the level editor, reload the canonical bank and check current saved level. **Text reverts:** update construction and refresh assignments from the exact table. **No live ads:** provider binaries are deliberately excluded; inspect your installed provider and fallback mode. **Scene reload exception:** retain owner-lifetime and generation guards on delayed callbacks. **WebGL fails:** inspect browser Console, compression headers and missing payloads.

CHAPTER 17

Support and final release checklist

Publisher: Szekipapa77. Support: szekipapa77@gmail.com. Include game title, exact Unity version, platform, first Console error and reproduction steps. Never send passwords/tokens.

1. Clean-import the standalone project and native package.
2. Play both scenes and replace a scene near startup.
3. Review tutorial, normal completion/retry and every helper activation plus effect.
4. Validate editor changes and test reskin text/colours in both layouts.
5. Rebuild affected Android/WebGL outputs.
6. Verify complete public APK bytes, actual browser demo, documentation and video links.
7. Keep licences and exclude all internal prompts, planning/capture/build/QA tooling from buyers.